

 Simon Leung



Recently, I watched a YouTube video titled ‘Using OpenAI GPT-4o Mini LLM for Contract Outlier Detection with KNIME.’ It got me thinking about whether the

same task could be accomplished without relying on a large language model (LLM). Specifically, I wondered if it would be possible to use a PDF parser to extract data from multiple PDF files, organize the extracted information into a dataframe, and then apply statistical or visualization techniques to identify any outliers in the dataset. This approach might offer a more lightweight and straightforward alternative for such tasks.

Import required library:

```
pip install PyPDF2
```

Load python libraries:

```
import numpy as np
import pandas as pd

#Plotting and Visualizations
import matplotlib.pyplot as plt
%matplotlib inline
import seaborn as sns
sns.set()
import os
import re
import PyPDF2
```

Check library versions:

```
import platform
print(f"Python version: {platform.python_version()}")
print(f"numpy version: {np.__version__}")
print(f"panda version: {pd.__version__}")
print(f"seaborn version: {sns.__version__}")
import matplotlib
print(f"matplotlib version: {matplotlib.__version__}")
print(f"PyPDF2 version: {PyPDF2.__version__}")
```

```
Python version: 3.10.14
numpy version: 1.26.4
panda version: 2.1.4
seaborn version: 0.13.2
matplotlib version: 3.7.5
PyPDF2 version: 3.0.1
```

I saved 100 PDF files into a folder by downloading the dataset (from [here](#)). Data extraction after parsing relies heavily on the use of regular expressions.

```
# Define the folder path containing the PDFs
folder_path = ".\Documents\contracts_pdf"

# Get list of PDF files in the folder
pdf_files = [f for f in os.listdir(folder_path) if f.endswith(".pdf")]

# Initialize a list to hold the extracted data
data = []

# Process each PDF file
for pdf_file in pdf_files:
    try:
        # Open the PDF in read-binary mode
        with open(os.path.join(folder_path, pdf_file), "rb") as file:
            reader = PyPDF2.PdfReader(file)
            # Extract text from all pages
            text = ""
            for page in reader.pages:
                extracted = page.extract_text()
                if extracted:
                    text += extracted + "\n"

            # If no text is extracted, skip this file
            if not text:
                print(f"Warning: No text extracted from {pdf_file}")
                continue

            # Preserve newlines for accurate parsing of multi-line patterns
            # Only normalize spaces within lines, not newlines
            lines = text.splitlines()
            text = "\n".join(line.strip() for line in lines if line.strip())

            # Extract fields using regular expressions
            # Agreement Number
            agreement_match = re.search(r"Investment Agreement (\w+)", text)
            agreement_number = agreement_match.group(1) if agreement_match else None
```

```

# Date
date_match = re.search(r"on (\d{4}-\d{2}-\d{2})", text)
date = date_match.group(1) if date_match else None

# Debtor Name
debtor_match = re.search(r'(.*), whose principal place of address is at',
debtor_name = debtor_match.group(1).strip() if debtor_match else None

# Creditor Name
creditor_match = re.search(r'(.*), of legal age, \(\hereinafter referred to
creditor_name = creditor_match.group(1).strip() if creditor_match else None

# Communication Method and Detail
comm_pattern = r"Preferred means of communication:.*?via (\w+)\s*\n\s*([^\n
comm_match = re.search(comm_pattern, text, re.DOTALL)
if comm_match:
    communication_method = comm_match.group(1)
    communication_detail = comm_match.group(2).strip()
else:
    communication_method = None
    communication_detail = None

# Creditor Payment Amount
payment_match = re.search(r"in the amount of ([\d\.]+) euros", text)
payment_amount = payment_match.group(1) if payment_match else None

# Investment Product
product_match = re.search(r'the investment product "(.+?)"', text)
investment_product = product_match.group(1) if product_match else None

# Store the extracted data in a dictionary
data.append({
    "Agreement Number": agreement_number,
    "Date": date,
    "Debtor Name": debtor_name,
    "Creditor Name": creditor_name,
    "Communication Method": communication_method,
    "Communication Detail": communication_detail,
    "Payment Amount": payment_amount,
    "Investment Product": investment_product,
    "PDF_File_Name": pdf_file
})

except Exception as e:
    print(f"Error processing {pdf_file}: {str(e)}")
    continue

# Create a pandas DataFrame from the extracted data
df = pd.DataFrame(data)

# Create a pandas DataFrame
df = pd.DataFrame(data)

```

```

# Convert data types
df['Payment Amount'] = pd.to_numeric(df['Payment Amount'], errors='coerce')
df['Date'] = pd.to_datetime(df['Date'], format='%Y-%m-%d', errors='coerce')

# Display the DataFrame
print("Extracted Data:")
print(df)

# Optionally, save the DataFrame to a CSV file
df.to_csv("investment_agreements_data.csv", index=False)
print("Data saved to 'investment_agreements_data.csv'")

```

Extracted Data:

	Agreement Number	Date	Debtor Name	Creditor Name \
0	C000000	2016-08-06	Bank A GmbH	Alois Berger
1	C000001	2016-02-11	Bank A GmbH	Michaela Schultz
2	C000002	2016-01-15	Bank A GmbH	Rotraut Grnwald
3	C000003	2016-02-26	Bank A GmbH	Helga Heindl
4	C000004	2016-04-05	Bank A GmbH	Mira Gleich
..	...	...	...	...
95	C000095	2016-10-25	Bank A GmbH	Marita Henkel
96	C000096	2016-02-21	Bank A GmbH	Julius Kunzmann
97	C000097	2016-11-18	Bank A GmbH	Dagmar Grtner
98	C000098	2016-12-13	Bank A GmbH	Mijo Burk
99	C000099	2016-08-21	Bank A GmbH	Stephanie Nickel

	Communication Method	Communication Detail	Payment Amount \
0	email	alois.berger@provider.com .	16.16
1	email	michaela.schultz@provider.com .	1996.55
2	email	rotraut.grnwald@provider.com .	860.40
3	email	helga.heindl@provider.com .	1184.51
4	email	mira.gleich@provider.com .	799.06
..	...	...	...
95	email	marita.henkel@provider.com .	1275.83
96	email	julius.kunzmann@provider.com .	1337.68
97	email	dagmar.grtner@provider.com .	1295.12
98	email	mijo.burk@provider.com .	1041.96
99	email	stephanie.nickel@provider.com .	9.11

	Investment Product	PDF_File_Name
0	Fund Manager+	11001.pdf
1	P+B Investment	11004.pdf
2	Private Investment	11005.pdf
3	Gold Investment	11011.pdf
4	Private Investment	11012.pdf
..	...	...
95	Private Investment	11190.pdf
96	Fund Manager+	11191.pdf
97	Private Investment	11193.pdf

```
98 Private Investment 11195.pdf
99 Gold Investment 11199.pdf

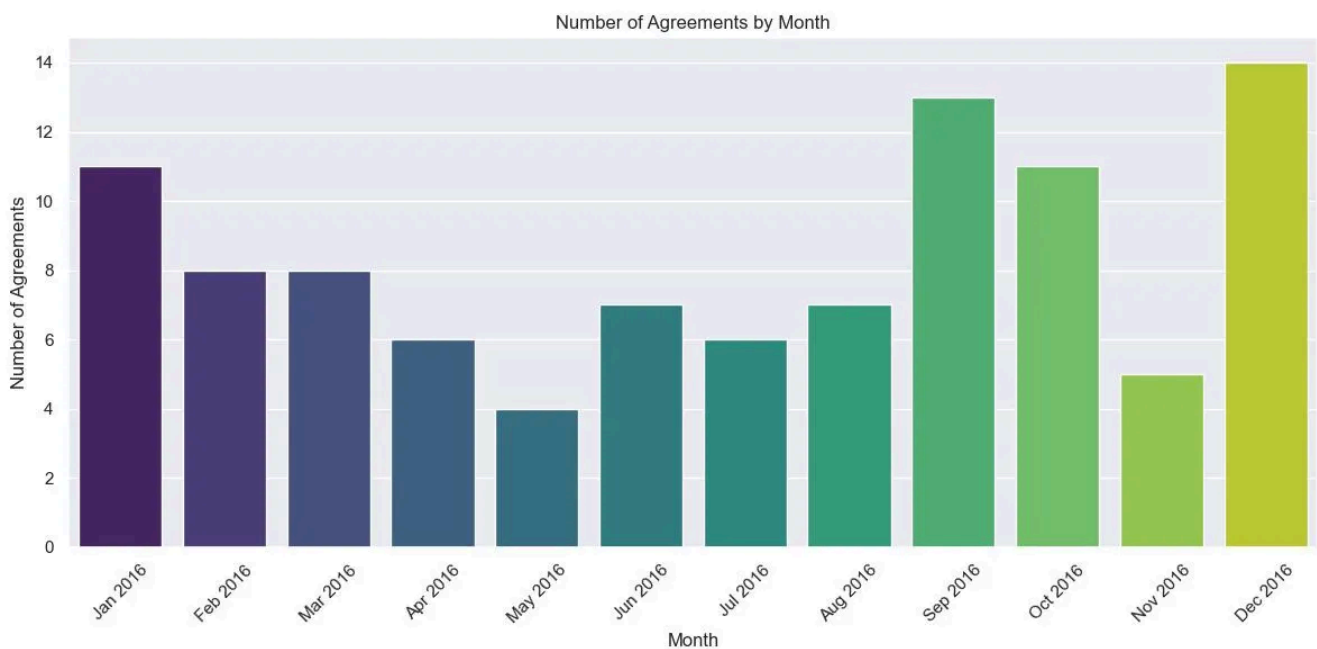
[100 rows x 9 columns]
```

Visualization:

```
### Agreements over time by month
# Resample the DataFrame by month and count agreements
monthly_counts = df.set_index('Date').resample('M').size().reset_index(name='Count')

# Create a 'Month' column with formatted labels
monthly_counts['Month'] = monthly_counts['Date'].dt.strftime('%b %Y')

# Create the bar plot
plt.figure(figsize=(12, 6)) # Set figure size for better visibility
sns.barplot(x='Month', y='Count', data=monthly_counts, palette='viridis')
plt.title('Number of Agreements by Month')
plt.xlabel('Month')
plt.ylabel('Number of Agreements')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



```

# Resample the DataFrame by month and count agreements per Investment Product
monthly_counts = df.groupby([df['Date'].dt.to_period('M'), 'Investment Product']).s

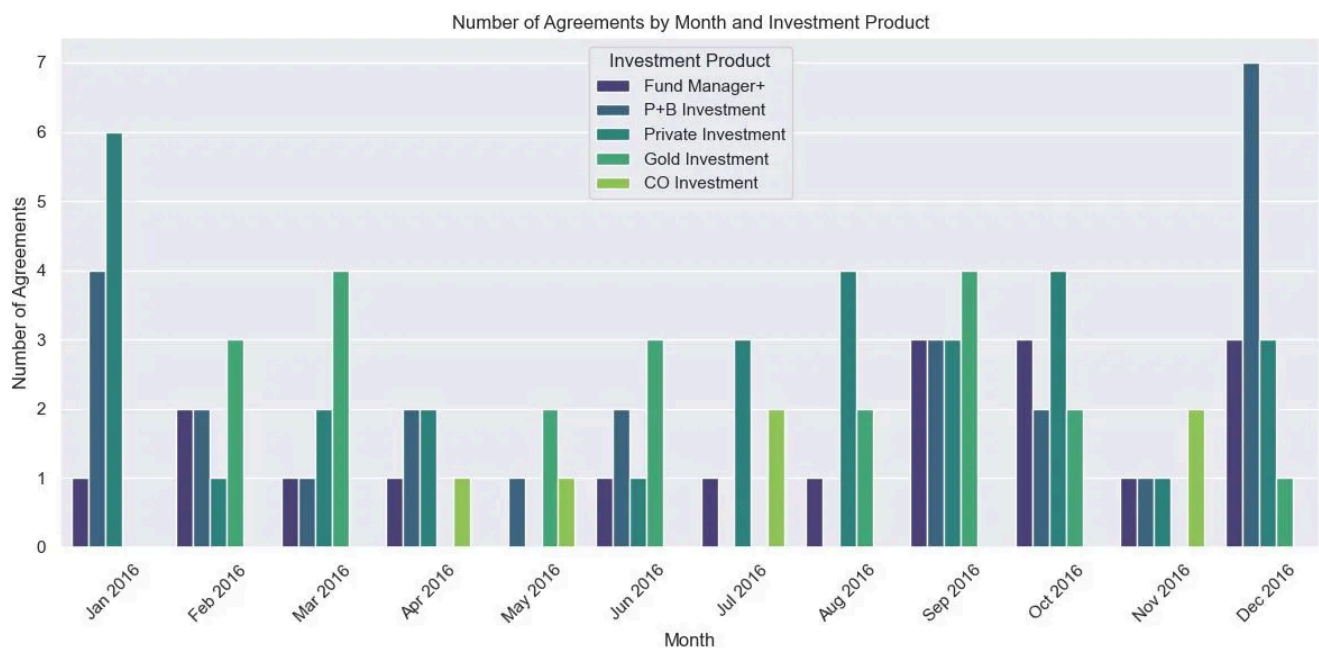
# Convert period to datetime for plotting
monthly_counts['Date'] = monthly_counts['Date'].dt.to_timestamp()

# Create a 'Month' column with formatted labels
monthly_counts['Month'] = monthly_counts['Date'].dt.strftime('%b %Y')

# Create the bar plot with hue
plt.figure(figsize=(12, 6)) # Set figure size for better visibility
sns.barplot(x='Month', y='Count', hue='Investment Product', data=monthly_counts, pa

plt.title('Number of Agreements by Month and Investment Product')
plt.xlabel('Month')
plt.ylabel('Number of Agreements')
plt.xticks(rotation=45)
plt.legend(title='Investment Product') # Add legend for clarity
plt.tight_layout()
plt.show()

```

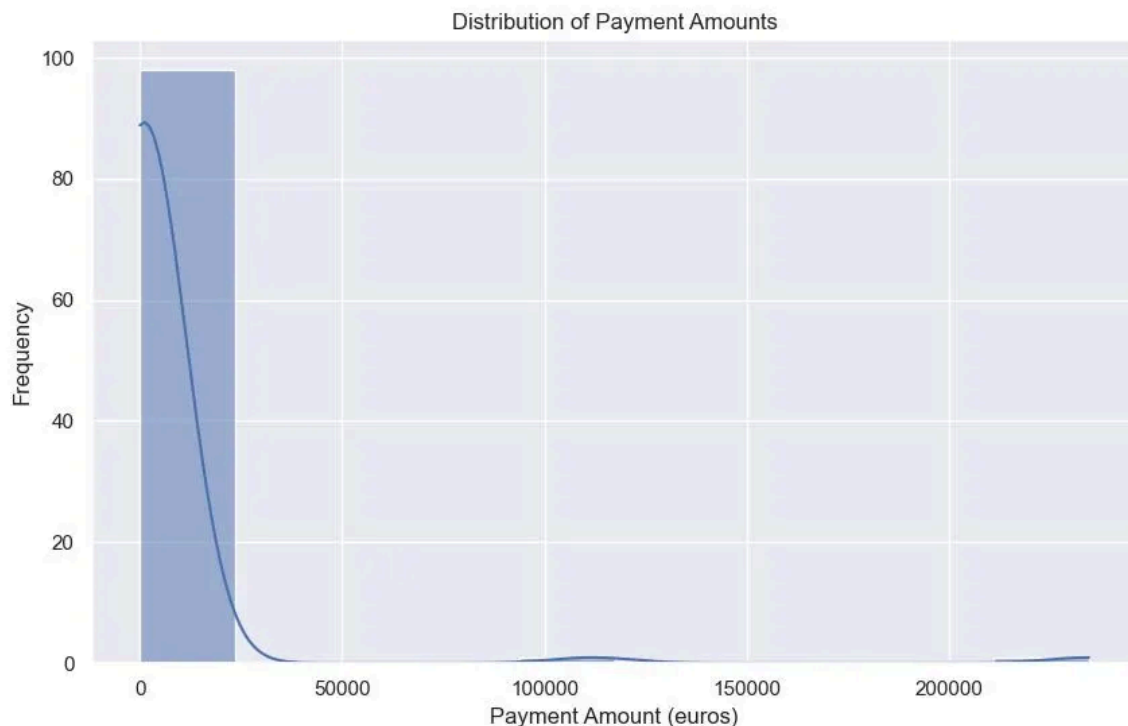


```

### Distribution of Payment Amounts
plt.figure(figsize=(10, 6))
sns.histplot(df['Payment Amount'], bins=10, kde=True)
plt.title('Distribution of Payment Amounts')
plt.xlabel('Payment Amount (euros)')

```

```
plt.ylabel('Frequency')
plt.show()
```



```
# Drop rows with missing values in 'Investment Product' or 'Payment Amount'
df_plot = df.dropna(subset=['Investment Product', 'Payment Amount'])

# Sort investment products by median payment amount
order = df_plot.groupby('Investment Product')['Payment Amount'].median().sort_value

# Create the scatter plot
plt.figure(figsize=(12, 6)) # Set figure size for readability
sns.stripplot(x="Investment Product", y="Payment Amount", data=df_plot, order=order)

# Add title and labels
plt.title('Scatter Plot of Payment Amount by Investment Product')
plt.xlabel('Investment Product')
plt.ylabel('Payment Amount (euros)')

# Rotate x-axis labels to prevent overlap
plt.xticks(rotation=45)

# Adjust layout to fit all elements
plt.tight_layout()
```



```
# Display the plot
plt.show()
```



Found 2 outliers, where 'Payment Amount' >5000 euros which belongs 'Private Investment'.

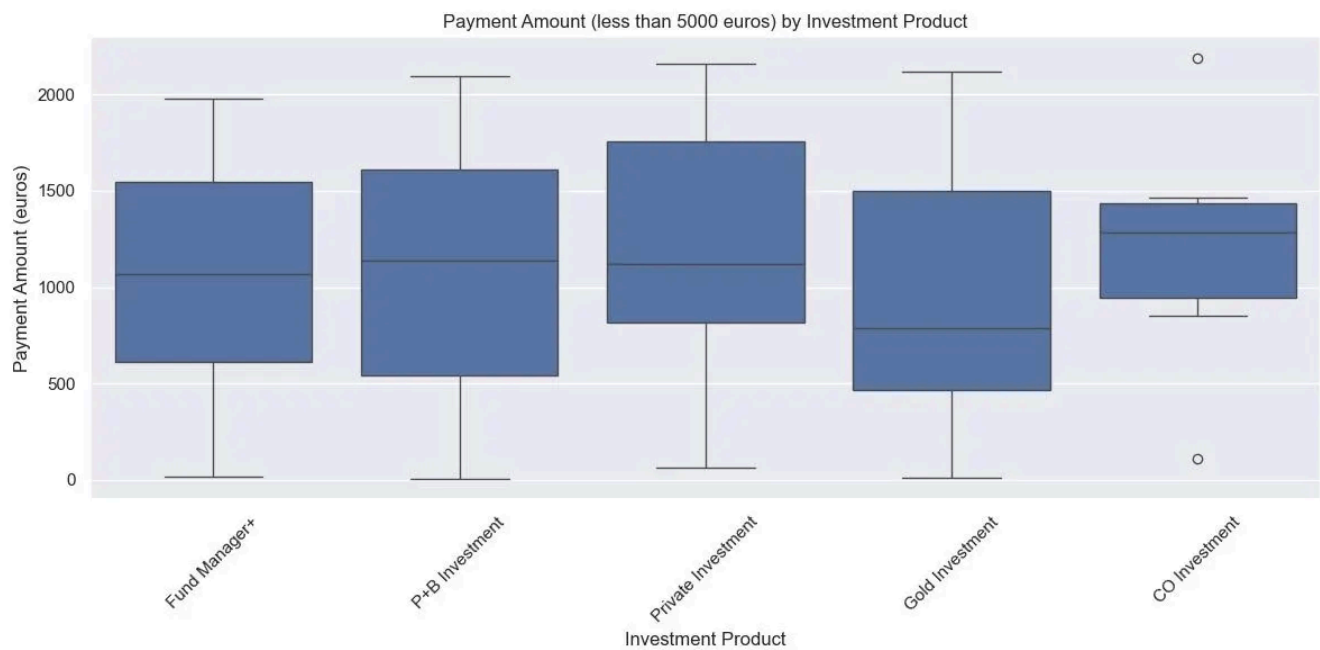
```
# Create the boxplot
df2 = df[df["Payment Amount"]<5000]
plt.figure(figsize=(12, 6)) # Set figure size for better visibility
sns.boxplot(x="Investment Product", y="Payment Amount", data=df2)

# Add title and axis labels
plt.title("Payment Amount (less than 5000 euros) by Investment Product")
plt.xlabel("Investment Product")
plt.ylabel("Payment Amount (euros)")

# Rotate x-axis labels for readability
plt.xticks(rotation=45)

# Adjust layout to prevent label cutoff
plt.tight_layout()

# Display the plot
plt.show()
```



For “CO Investment”, there’re 2 outliers under 5000 euros exist.

To find out which outliers agreement and pdf files:

```
# Initialize an empty list to store outliers
outliers = []

# Loop through each unique investment product
for product in df['Investment Product'].unique():
    # Filter data for the current product
    product_data = df[df['Investment Product'] == product]

    # Calculate quartiles and IQR
    Q1 = product_data['Payment Amount'].quantile(0.25)
    Q3 = product_data['Payment Amount'].quantile(0.75)
    IQR = Q3 - Q1

    # Define outlier bounds
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    # Identify outliers
    product_outliers = product_data[
        (product_data['Payment Amount'] < lower_bound) |
        (product_data['Payment Amount'] > upper_bound)
    ]

    # Add outliers to the list
    outliers.append(product_outliers)

# Combine all outliers into a single DataFrame
```

```
outliers_df = pd.concat(outliers)
```

```
print(outliers_df)
```

	Agreement Number	Date	Debtor Name	Creditor Name	\
15	C000015	2016-01-18	Bank A GmbH	Kai-Uwe Eger	
19	C000019	2016-08-27	Bank A GmbH	Cosima Brandl	
16	C000016	2016-11-27	Bank A GmbH	Dimitri Rhrs	
76	C000076	2016-07-13	Bank A GmbH	Manja Joseph	

	Communication	Method	Communication Detail	Payment Amount	\
15		email	kai-uwe.eger@provider.com .	234512.98	
19		email	cosima.brandl@provider.com .	111729.92	
16		email	dimitri.rhrs@provider.com .	111.39	
76		email	manja.joseph@provider.com .	2188.51	

	Investment Product	PDF_File_Name
15	Private Investment	11034.pdf
19	Private Investment	11042.pdf
16	CO Investment	11035.pdf
76	CO Investment	11160.pdf

Conclusions:

1. Using a Python script offers greater flexibility, allowing for customization and fine-tuning of the data extraction process. However, it heavily relies on the correct implementation of regular expressions, which can be complex and challenging to get right. This approach becomes particularly important given that the GPT-4o Mini LLM API is not freely available, making an alternative method essential for cost-effective data processing.
2. The PyPDF2 library is limited in its application as it only works on PDFs that are structured, non-scanned, and do not contain tables or images. This means that for scanned documents or PDFs with complex layouts, additional preprocessing steps — such as optical character recognition (OCR) or specialized parsing techniques — may be required to extract meaningful data.

Choosing the Right Library

- For structured PDFs (text-based, well-formatted): `pdfplumber` or `PDFMiner.six`

- For PDFs with tables: pdfplumber
- For basic text extraction from structured PDFs: PyPDF2
- For unstructured/scanned PDFs: pdf2image + Tesseract OCR
- For metadata extraction: Apache Tika

Library	Works with Structured PDFs	Works with Unstructured PDFs	OCR Support	Preserves Layout	Handles Tables	Speed
PyPDF2	✔ Yes	✘ No	✘ No	✘ No	✘ No	★★★★
pdfplumber	✔ Yes	✘ No	✘ No	✔ Yes	✔ Yes	★★★
PDFMiner.six	✔ Yes	✘ No	✘ No	✔ Yes	✘ No	★★
pdf2image	✘ No	✔ Yes	✔ Yes	✘ No	✘ No	★★★
Tesseract OCR	✘ No	✔ Yes	✔ Yes	✘ No	✘ No	★★
Apache Tika	✔ Yes	✔ Yes	✔ Limited	✘ No	✘ No	★★
...						

Thank you for reading.

Written by Simon Leung

229 followers · 4 following

A fervent enthusiast, fueled by an insatiable curiosity for STEM (Science, Technology, Engineering, and Mathematics) disciplines.



